

Performance Testing

Date	03 October 2026
Team ID	SWTID-2026-9455
Project Name	ComicCraft - AI Comic Story Creator using Gemini Models
Maximum Marks	5 Marks

Scope note: a full real-model run (Gemini + Stable Diffusion) was also completed by the team on 30 Sep 2026 and worked correctly; its timing was not recorded. For the load tests below, AI calls were replaced by the built-in offline mock mode (COMICCRAFT MOCK=1), so these results measure ComicCraft's own overhead (routing, templates, image drawing with Pillow, layout, PDF export) and not Gemini or Stable Diffusion latency. Real-model timings must be measured with the same script on your machine.

Step 1: Testing Overview

Field	Details
Testing Tool Used	scripts/load_test.py (Python, httpx + threads; closed-loop virtual users without think time); psutil sampling for CPU and memory
Type of Testing	Load testing (4 scenarios) with a saturation observation on CPU
Target Module / API	GET /, GET /health, POST /generate-comic/json, POST /generate (full workflow)
Test Environment	Local Linux sandbox, 1 vCPU, 4.2 GB RAM, one Uvicorn process, mock AI mode
Test Date	03 October 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Homepage load: repeated GET /	10	10	All responses 200, page served in well under 2 s
2	Health-check flood: GET /health	50	10	No errors with 50 concurrent users
3	JSON API: POST /generate-comic/json (full workflow, mock AI)	5	15	All 200, valid layout and PDF each time
4	Form workflow: POST /generate (render preview + PDF, mock AI)	10	15	All 200, avg < 2 s, no errors

Measured results per scenario

Scenario	Requests	Errors	Avg (s)	p95 (s)	Max (s)	Throughput (req/s)
1. GET / (10 users)	10979	0	0.0089	0.0115	0.067	1097.2
2. GET /health (50 users)	10998	0	0.042	0.0639	0.3548	1096.4
3. POST /generate-comic/json (5 users)	144	0	0.522	0.5862	0.6538	9.5
4. POST /generate (10 users)	124	0	1.2237	1.4195	1.5771	8.0

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2237 s (full workflow); 0.0089 s (homepage)	Pass	Scenario 4, 10 users; mock AI
2	Response Time (Max)	< 5 seconds	1.5771 s	Pass	Worst case in scenario 4
3	Throughput (Req/sec)	> 5 req/s (full workflow)	8.0 req/s (full workflow); 1097.2 req/s (homepage)	Pass	Single process, single vCPU
4	Error Rate	< 1%	0% in all four scenarios	Pass	22245 requests, 0 errors
5	CPU Utilization	< 80%	97.8% avg, 100% peak	Fail	Saturated 1-vCPU machine under back-to-back load with no think time
6	Memory Utilization	< 80%	3.17% peak (133.0 MB of 4.2 GB)	Pass	Server process only; excludes the Stable Diffusion model

Step 4: Observations & Analysis

- The application overhead is small: pages are served in about 9 ms and a full mock comic (5 images, layout and 6-page PDF) takes about 1 second, with no errors in 22,245 requests.
- CPU was the bottleneck: a single vCPU reached 100% under 10 users hammering /generate without pauses. This is expected for a closed-loop test and is recorded honestly as a Fail against the 80% target; real users pause between requests. Adding Uvicorn workers or cores raises throughput.
- With real models, total time will be dominated by Gemini (seconds) and by Stable Diffusion image generation (seconds per image on a GPU, much longer on CPU). Image inference is deliberately serialised with a lock, so concurrent real users queue for the GPU. A background job queue is listed as future scope.
- To measure real-model timings: run the server without COMICCRAFT MOCK and use python scripts/load_test.py --path /generate --method POST --users 1 --duration 120.

Step 5: Screenshots / Evidence

Raw console output of the four runs (from scripts/load_test.py) and the resource sampling of run 4:

```
{
  "path": "/",
  "method": "GET",
  "users": 10,
  "duration_s": 10.0,
  "requests": 10979,
  "errors": 0,
  "error_rate_pct": 0.0,
  "avg_s": 0.0089,
  "p95_s": 0.0115,
  "max_s": 0.067,
  "throughput_rps": 1097.2
}
{
  "path": "/health",
  "method": "GET",
  "users": 50,
  "duration_s": 10.0,
  "requests": 10998,
  "errors": 0,
  "error_rate_pct": 0.0,
  "avg_s": 0.042,
  "p95_s": 0.0639,
  "max_s": 0.3548,
  "throughput_rps": 1096.4
}
{
  "path": "/generate-comic/json",
  "method": "POST",
  "users": 5,
  "duration_s": 15.2,
  "requests": 144,
  "errors": 0,
  "error_rate_pct": 0.0,
  "avg_s": 0.522,
  "p95_s": 0.5862,
  "max_s": 0.6538,
  "throughput_rps": 9.5
}
{
  "path": "/generate",
  "method": "POST",
  "users": 10,
  "duration_s": 15.5,
  "requests": 124,
  "errors": 0,
  "error_rate_pct": 0.0,
  "avg_s": 1.2237,
  "p95_s": 1.4195,
  "max_s": 1.5771,
  "throughput_rps": 8.0
}
{
  "cores": 1,
  "sys_cpu_avg": 97.8,
  "sys_cpu_peak": 100.0,
  "proc_cpu_peak_pct_of_1core": 101.9,
  "mem_peak_pct_of_total_ram": 3.17,
  "rss_peak_mb": 133.0,
  "total_ram_gb": 4.2
}
```